

DLKS-MQTT: A Lightweight Key Sharing Protocol for Secure IoT Communications

A PROJECT REPORT

Submitted by

Harshil Srivastava (24BCY10399)

Sankalp Tripathi (24BCY10005)

Mayank Singh (24BCY10144)

Palla Pranavi (24BCY10131)

Rhythm Sharma (24BCY10103)

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

**COMPUTER SCIENCE AND ENGINEERING
(Cyber Security and Digital Forensics)**



**SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL
INTELLIGENCE (SCAI)**

VIT BHOPAL UNIVERSITY

**KOTHRIKALAN, SEHORE
MADHYA PRADESH - 466114**

**VIT BHOPAL UNIVERSITY, KOTHRI KALAN,
SEHORE
MADHYA PRADESH – 466114**

BONAFIDE CERTIFICATE

Certified that this project report titled “DLKS-MQTT: A Lightweight Key Sharing Protocol for Secure IoT Communications” is the Bonafide work of Harshil Srivastava (24BCY10399), Sankalp Tripathi (24BCY10005), Mayank Singh (24BCY10144), Palla Pranavi (24BCY10131), and Rhythm Sharma (24BCY10103) who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

DIVISION HEAD
Dr. D. Saravanan
Division Head,
School of Computing
Science Engineering and
Artificial Intelligence
(SCAI)VIT BHOPAL
UNIVERSITY

PROGRAM CHAIR
Dr. Adarsh Patel
Assistant Professor,
School of Computing
Science Engineering and
Artificial Intelligence
(SCAI)VIT BHOPAL
UNIVERSITY

PROJECT GUIDE
Dr Hemraj .S. Lamkuche
Assistant Professor,
School of Computing
Science Engineering and
Artificial Intelligence
(SCAI)VIT BHOPAL
UNIVERSITY

ACKNOWLEDGEMENT

First and foremost I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I wish to express my heartfelt gratitude to **Dr D. Saravanan**, Division Head and **Dr. Adarsh Patel**, Program Chair, Cyber Security and Digital Forensics for much of his valuable support encouragement in carrying out this work.

I would like to thank our internal guide **Dr Hemraj .S.Lamkuche** for continually guiding and actively participating in my project, giving valuable suggestions to complete the project work.

I would like to thank all the technical and teaching staff of the School Computing Science and Engineering, who extended directly or indirectly all support.

LIST OF ABBREVIATIONS

- **IoT**: Internet of Things
- **MQTT**: Message Queuing Telemetry Transport
- **DLKS-MQTT**: Dynamic Lightweight Key Sharing for MQTT
- **LCG**: Linear Congruential Generator
- **MitM**: Man-in-the-Middle
- **DDoS**: Distributed Denial of Service
- **ABE**: Attribute-Based Encryption

LIST OF FIGURES AND GRAPHS

FIGURE NO.	TITLE	PAGE NO.
1	CPU Energy consumption comparison	26
2	Execution time comparison	27
3	Baseline MQTT and DLKS-MQTT	28

ABSTRACT

The rapid expansion of the Internet of Things (IoT) across critical sectors such as healthcare, smart cities, and industrial automation has introduced significant security challenges, particularly for resource-constrained devices. The Message Queuing Telemetry Transport (MQTT) protocol, while widely adopted for its lightweight nature, remains inherently vulnerable to sophisticated cyber threats including eavesdropping, message forging, and Man-in-the-Middle (MitM) attacks. Conventional cryptographic techniques present challenges of increased computational and power intensity that modern IoT devices cannot support. This project addresses this gap by implementing the Dynamic Lightweight Key Sharing for MQTT (DLKS-MQTT) mechanism, designed to provide a balance between robust security and operational efficiency.

The system is built on a dynamic authentication framework that integrates ephemeral key generation, a minimalistic handshake protocol, and lightweight cryptographic operations. The core of the detection and protection capability lies in the use of a 128-bit ephemeral session key generated via an optimized Linear Congruential Generator (LCG). Authentication procedures are embedded directly into the standard MQTT "connect" and "connack" messages, utilizing lightweight BLAKE2s hashing to guarantee message integrity without adding additional communication overhead. The implementation follows a client-broker architecture where the session keys are transient, providing unique security for each communication session to eliminate long-term vulnerabilities.

Extensive performance evaluations of the functional prototype were conducted using the Cooja simulator with realistic IoT network configurations. The results demonstrated a high degree of efficiency, achieving a CPU energy consumption of only 0.000002 mJ and an execution time of 3.75 seconds. Communication overhead was successfully minimized to 60 bytes, outperforming existing methods such as SMQTT and ICP-ABE. This outcome confirms the tool's effectiveness for real-time secure IoT communications, delivering a practical security solution that empowers resource-constrained devices to maintain data integrity and confidentiality in hostile network environments.

TABLE OF CONTENTS (**SPECIMEN**)

CHAPTER NO.	TITLE	PAGE NO.
	List of Abbreviations	4
	List of Figures and Graphs	5
	Abstract	6
1	CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE Introduction Motivation for the work Introduction to MQTT and Security Techniques Problem Statement Objective of the work Organization of the project Summary	11 11 11 11 12 12 13 13

2	CHAPTER-2: RELATED WORK INVESTIGATION Introduction Core Area: Lightweight IoT Security and Key Management Secure MQTT (SMQTT)s Improved Ciphertext Policy-Attribute-Based Encryption (ICP-ABE) Dynamic Lightweight Authentication for MQTT (DLA-MQTT) Key Schedule Algorithm (KSA)-PRESENT Pros and Cons of Stated Approaches Issues and Observations from Investigation Summary	14 14 14 14 15 15 15 15 15 16
3	CHAPTER-3: REQUIREMENT ARTIFACTS Introduction Hardware and Software Requirements Specific Project Requirements Data Requirement	17 17 17 18 18

	Functional Requirement	18
	Performance and Security Requirement	18
	Look and Feel Requirements	18
	Summary	19
4	CHAPTER-4: DESIGN METHODOLOGY AND ITS NOVELTY	20
	Methodology and Goal	20
	Functional Modules Design and Analysis	20
	Software Architectural Designs	21
	Subsystem Services (LCG and Hashing)	22
	User Interaction/Interface Designs	22
	Summary	23
5	CHAPTER-5: TECHNICAL IMPLEMENTATION & ANALYSIS	23
	Technical Coding and Code Solutions	23
	Performance Analysis (Graphs/Charts Interpretation)	24
	Test and validation	27
	Summary	

6	CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY	29
	Outline	29
	Key Implementation Outlines of the DLKS-MQTT	29
	Significant Project Outcomes	29
	Project Applicability on Real-world Applications	30
	Inference	30
7	CHAPTER-7: CONCLUSIONS AND RECOMMENDATION	31
	Outline	31
	Limitations/Constraints of the System	31
	Future Enhancements	31
	Inference	32
	References	33

CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE

- **1.1 Introduction**

Internet of Things (IoT) has fundamentally transformed modern industrial and domestic landscapes, including critical sectors such as healthcare, smart homes, automotive, smart factories, smart grids, and smart cities. While these environments represent the pinnacle of digital interconnectivity, they also introduce significant security risks. Malicious actors often target insecure IoT devices to launch massive Distributed Denial of Service (DDoS) attacks, such as the infamous Mirai botnet incident. These security breaches lead to substantial losses in computational capacity, memory, and power consumption for the affected devices.

Message Queuing Telemetry Transport (MQTT) serves as the primary, most widely used lightweight messaging protocol for these IoT applications. Despite its popularity, MQTT is inherently insecure by default, lacking robust built-in mechanisms to prevent eavesdropping, message forging, and Man-in-the-Middle (MitM) attacks. Conventional cryptographic techniques, while technically sound, often fail in the IoT context due to the high computational and power intensity required—demands that modern resource-constrained IoT devices cannot support.

- **1.2 Motivation for the Work**

The motivation for this project stems from the critical need to optimize security solutions for the IoT domain. Existing methods often lack the flexibility required to adapt to frequently changing device states, network topologies, or emerging threats. Furthermore, state-of-the-art techniques such as post-quantum cryptography and low-overhead authentication remain insufficiently explored for practical deployment on edge devices.

This work is motivated by the desire to bridge the gap between complex cryptographic mechanisms and the practical limitations of IoT hardware. By developing the Dynamic Lightweight Key Sharing for MQTT (DLKS-MQTT) mechanism, this project seeks to achieve a balance between high-level security and operational resource efficiency. The implementation of ephemeral key generation and minimalistic handshake protocols serves as a proactive response to the energy and memory constraints of modern IoT nodes.

- **1.3 [Introduction to the Project including Techniques]**

The DLKS-MQTT mechanism is a novel security framework that enhances the standard MQTT protocol by integrating ephemeral key generation, streamlined authentication, and lightweight

cryptographic operations. The system establishes a secure data exchange environment while maintaining compatibility with the existing MQTT architecture.

The project utilizes several sophisticated yet lightweight techniques:

- **Ephemeral Session Keys:** Session-specific 128-bit keys are generated using a Linear Congruential Generator (LCG) to eliminate long-term vulnerabilities.
- **Minimalistic Handshake:** Security features are embedded directly into standard MQTT "connect" and "connack" messages to avoid additional communication overhead.
- **Lightweight Hashing:** The BLAKE2s or SipHash algorithms are leveraged to ensure message integrity and authenticity without burdening the device's CPU.
- **Public Key Encryption:** A broker's public key is used during the initial connection to secure the transmission of the session key.

- **1.4 Problem Statement**

The fundamental problem addressed by this project is the high vulnerability of the MQTT protocol in resource-constrained IoT environments. Standard MQTT does not provide built-in security for authentication or data confidentiality. While traditional encryption layers (such as TLS) exist, they are often too computationally expensive and power-intensive for low-power sensors and actuators. Consequently, a significant research gap exists for adaptive, lightweight cryptographic solutions that can provide robust protection against replay and MitM attacks without degrading the performance of the IoT network.

- **1.5 Objective of the Work**

The primary objectives of this project implementation include:

- **Dynamic Key Sharing:** Develop a mechanism that generates ephemeral session keys with minimal computational and energy requirements.
- **Vulnerability Mitigation:** Uncover and mitigate threats such as eavesdropping, replay attacks, and MitM attacks in MQTT-based networks.
- **Performance Evaluation:** Evaluate the protocol's computational efficiency, scalability, and security robustness under diverse IoT conditions using the Cooja simulator.
- **Resource Optimization:** Achieve a balance between high security and resource efficiency by minimizing communication and computational overhead.

- **1.6 Organization of the Project**

The report is organized into seven comprehensive chapters following the VIT Bhopal project report standard:

- **Chapter 1** provides an overview of the project, including motivation, objectives, and problem statement.
- **Chapter 2** investigates related works and compares existing lightweight security methods.
- **Chapter 3** defines the hardware and software requirement artifacts for the simulation environment.
- **Chapter 4** describes the design methodology of the DLKS-MQTT protocol and its architectural novelty.
- **Chapter 5** presents the technical implementation, pseudocode, and detailed analysis of experimental results.
- **Chapter 6** discusses the project outcomes and its applicability in real-world IoT scenarios.
- **Chapter 7** concludes the report with recommendations for future enhancements.

- **1.7 Summary**

In summary, Chapter 1 has established the foundational context for the DLKS-MQTT project. It highlighted the critical security flaws in standard MQTT and the necessity for a lightweight, dynamic solution suitable for resource-constrained IoT devices. The subsequent chapters will detail the specific methodology and simulation results that prove the efficacy of this proposed approach in maintaining a balance between high security and low resource consumption

CHAPTER-2: RELATED WORK INVESTIGATION

- **2.1 Introduction**

The rapid integration of the Internet of Things (IoT) into diverse industrial and consumer sectors has revolutionized data collection and utilization. However, the foundational communication protocols, specifically the **Message Queuing Telemetry Transport (MQTT)**, were originally designed for low-bandwidth and resource-constrained environments rather than high-security scenarios. As a result, standard MQTT lacks built-in security features, leaving it highly vulnerable to eavesdropping, message forgery, and Man-in-the-Middle (MitM) attacks. This chapter investigates existing research into lightweight cryptographic enhancements, authentication frameworks, and the prevailing research gaps in IoT security.

- **2.2 Core Area: Lightweight IoT Security and Key Management**

Security solutions for IoT must be specifically optimized to account for limited processing power, memory, and energy sources. Traditional cryptographic methods are often too power-intensive for these environments. Key research in this core area includes:

- **Lightweight Cryptography:** Recent publications emphasize the feasibility of simplified encryption and authentication schemes to protect networks without overloading device resources.
- **Authentication Mechanisms:** Innovative frameworks have been proposed using blockchain for device authenticity or lightweight Public Key Infrastructures (PKIs) for manageable certificate systems.
- **Dynamic Key Management:** Systems that account for the frequently changing states and topologies of IoT networks are essential for maintaining long-term security.

- **2.3 Existing Approaches/Methods**

Several distinct methodologies have been developed to address MQTT's inherent vulnerabilities. These methods serve as the baseline for evaluating the performance of the proposed DLKS-MQTT protocol.

2.3.1 Secure MQTT (SMQTT)

SMQTT is a security-focused extension of the standard protocol. While it provides a high level of protection, it is characterized by high communication overhead, requiring 128 bytes per message. Furthermore, it exhibits the highest average energy consumption (0.00177 mJ) and CPU energy consumption (0.000472 mJ) among the reviewed methods. Its execution time is approximately 2.8 seconds, making it computationally intensive for many low-power applications.

2.3.2 Improved Ciphertext Policy-Attribute-Based Encryption (ICP-ABE)

This approach utilizes attribute-based encryption to secure data. While robust, it incurs a moderate computational overhead of 305.49 units. Its communication overhead is 64 bytes, and it requires an execution time of 1.6 seconds. This method is safer than standard MQTT but still more demanding than modern lightweight alternatives.

2.3.3 Dynamic Lightweight Authentication for MQTT (DLA-MQTT)

DLA-MQTT is a modern approach that performs closely to the proposed DLKS-MQTT protocol. It features a low communication overhead of 60 bytes and a computational overhead of 280 units. It consumes 0.0015 mJ of energy with an execution time of 3.75 seconds.

2.3.4 Key Schedule Algorithm (KSA)-PRESENT

This method employs the PRESENT cipher for security. It demonstrates moderate energy consumption (0.0016 mJ) but suffers from a significant execution time of 1.8 seconds and a higher communication overhead of 80 bytes compared to more optimized solutions.

- 2.4 Pros and Cons of Stated Approaches

Method	Advantages	Disadvantages
SMQTT	High security and standardization.	Highest energy drain and heavy communication overhead.
ICP-ABE	Robust attribute-based access control.	Higher execution time (1.6s) than lightweight alternatives.
DLA-MQTT	Very efficient with low latency.	Slightly higher energy use compared to DLKS-MQTT.
KSA-PRESENT	Established cipher logic.	Relatively slow execution and higher message overhead.

- 2.5 Issues and Observations from Investigation

The primary issue observed in existing literature is the trade-off between security robustness and resource efficiency. While much research has been successful in creating lightweight solutions, many still fail to respond to dynamic security threats in a context-aware manner. Investigating these improvements reveals a research gap in domain-specific, adaptive protocols that minimize computational intensity while maintaining resistance to cryptanalytic and replay attacks.

- **2.6 Summary**

Chapter 2 has provided a detailed review of the current landscape of MQTT security. By analyzing methods such as SMQTT, ICP-ABE, and DLA-MQTT, it is evident that while progress has been made, there is still a significant need for a more optimized solution. The proposed **DLKS-MQTT** seeks to fill this gap by leveraging ephemeral key generation and minimalistic handshakes to outperform these existing benchmarks in both efficiency and security.

CHAPTER-3: REQUIREMENT ARTIFACTS

- **3.1 Introduction**

The development and evaluation of the **DLKS-MQTT** protocol require a specialized simulation environment capable of emulating realistic IoT network behaviors. To ensure the protocol is suitable for resource-constrained devices, the requirements are focused on low-power hardware profiles and efficient software stacks. This chapter outlines the hardware, software, and specific functional artifacts necessary to replicate the experimental setup and validate the security performance of the proposed protocol.

- **3.2 Hardware and Software Requirements**

3.2.1 Hardware Requirements (Simulated Profiles)

The project utilizes simulated hardware profiles to test the protocol's impact on energy and memory.

- **IoT Nodes (Sky Motes):** 30 client nodes are modeled after Sky motes, which are industry-standard for low-power research due to their limited MSP430 microcontrollers and IEEE 802.15.4 radio interfaces.
- **MQTT Broker:** One node is designated as the central MQTT broker, responsible for managing subscriptions and verifying authentication payloads.
- **Attacker Nodes:** Two dedicated nodes are configured to perform malicious activities, specifically simulating **Man-in-the-Middle (MitM)** and replay attack vectors.
- **Mesh Network Topology:** The devices are arranged in a mesh configuration within a $200\text{m} \times 200\text{m}$ square area.
- **Transmission Range:** Each node maintains a transmission range of 50 m to ensure multi-hop communication scenarios are tested.

3.2.2 Software Requirements

The software stack is chosen for its compatibility with resource-aware IoT development.

- **Operating System: Contiki OS**, an open-source operating system designed specifically for the Internet of Things with a small memory footprint.
- **Simulator: Cooja Simulator**, which allows for the cross-layer simulation of sensor networks and provides detailed energy consumption metrics.
- **Messaging Protocol: MQTT Version 3.1.1**, modified to support the DLKS-MQTT handshake enhancements.

- **Programming Language:** C, used for implementing the cryptographic modules and network logic within the Contiki environment.
- **3.3 Specific Project Requirements**

3.3.1 Data Requirement

- **Payload Size:** The system is designed to handle a base payload size of 128 bytes, which includes the data and the DLKS-MQTT security overhead.
- **Key Size:** 128-bit ephemeral session keys are required for each session to meet modern cryptographic standards.
- **Communication Frequency:** Nodes are required to transmit one message every two minutes to simulate typical telemetry reporting.

3.3.2 Functional Requirement

- **Dynamic Key Generation:** The system must generate a new, unique session key for every connection request using a Linear Congruential Generator (LCG).
- **Handshake Integration:** Security fields (key, nonce, and hash) must be embedded within the existing MQTT CONNECT and CONNACK packet structures.
- **Broker Authentication:** The broker must possess the capability to decrypt incoming payloads using a private key and verify the authenticity of the device.

3.3.3 Performance and Security Requirement

- **Network Latency:** The total time for a secure session establishment must remain under 500 ms to ensure usability in real-time applications.
- **Integrity Protection:** The system must utilize the **BLAKE2s-128-bit** hash function to detect any unauthorized modifications to the data in transit.
- **Anti-Replay Mechanism:** Every connection request must include a unique **Nonce (N)** to prevent attackers from re-sending previously captured packets.
- **Energy Constraints:** The protocol must minimize CPU cycles and energy consumption to maintain the longevity of battery-powered nodes.

3.3.4 Look and Feel Requirements

- **Transparency:** For the end-user/application layer, the security enhancements should remain transparent, following standard MQTT subscribe/publish workflows.

- **Status Reporting:** The system should provide feedback through the CONNACK message indicating whether authentication was successful or failed.

3.3.5 summary

Chapter 3 has defined the technical foundation required to implement the DLKS-MQTT project. By specifying the **Sky mote** profiles, the **Cooja/Contiki** software environment, and the 128-bit security parameters, this requirement phase ensures that the subsequent design and implementation phases are grounded in a realistic, resource-constrained IoT context

CHAPTER-4 DESIGN METHODOLOGY AND ITS NOVELTY

● 4.1 Methodology and Goal

The primary goal of this work is to study and implement a lightweight security-oriented communication mechanism inspired by the DLKS-MQTT framework used in IoT systems. The objective is to evaluate how pseudo-random key generation and hashing techniques can improve data integrity and security while maintaining computational efficiency. The referenced research highlights that lightweight cryptographic operations combined with ephemeral key generation enhance confidentiality and protect communication from attacks such as replay or interception.

To achieve this objective, an experimental methodology based on practical implementation was followed. Three major program components were developed using the C programming language. Two separate C files were created: one representing a base implementation and another representing a refined or real implementation. The base program executed simple pseudo-random key generation and hashing with limited optimization. The improved program introduced better structural organization, refined LCG-based number generation, and enhanced processing flow.

In addition, a keylogger-based module was developed to simulate realistic input streams. This module served as an input data provider for both implementations, enabling controlled testing under identical conditions. Both program versions were executed repeatedly, and outputs were recorded to observe execution characteristics and data behavior.

Performance metrics including execution flow consistency, generated values, and processing patterns were documented. These recorded values were then converted into graphical representations for comparative analysis between base and improved implementations.

This methodology combines theoretical understanding from research literature with hands-on implementation, comparative testing, and visualization. It ensures systematic evaluation of lightweight cryptographic concepts through measurable experimentation, validating practical behavior alongside theoretical design goals.

● 4.2 Functional Module and Design Analysis

The system is organized into distinct functional modules that collectively simulate lightweight cryptographic processing. Each module has a specific role, ensuring clarity of design and easier evaluation of performance.

The first module is the Key Generation Module. This unit uses Linear Congruential Generator logic to produce pseudo-random sequences that simulate temporary session keys. Such transient keys improve security by reducing exposure duration and limiting predictability. This module exists in both base and enhanced program files, enabling comparative behavior analysis.

The second module is the Hash Processing Module. Its responsibility is to transform generated data into fixed-length values for integrity verification. Hashing ensures that modifications to data can be detected and allows structured output validation. The lightweight nature of hashing operations ensures that system performance remains stable during repeated execution.

The third module is the Input Simulation Module implemented through the keylogger-based component. This module captures input sequences that act as dynamic data feeds for testing. It helps replicate realistic execution conditions, producing variable datasets for module evaluation.

Design analysis reveals a sequential interaction structure where keys are generated, hashed, and evaluated using simulated input. Modular separation allows independent modification without affecting overall execution flow. Comparative execution demonstrated improved stability and structured output behavior in the refined implementation, validated through graphical comparison.

Overall, the modular design promotes flexibility, scalability, and clarity in functional responsibilities, enabling efficient experimental analysis of lightweight cryptographic workflows.

● 4.3 Software Architectural Design

The software architecture follows a layered structural model that separates computation, logic processing, and evaluation. This layered approach improves maintainability, scalability, and clarity of execution.

The lowest level is the Processing Layer, containing the core C programs responsible for executing algorithms. Both base and improved implementations reside here, performing key generation and hashing computations directly through processor interaction.

Above this lies the Security Logic Layer, which organizes algorithmic flow involving LCG-based key generation and hash verification. This layer reflects conceptual security design where transient keys and lightweight operations are structured to maintain efficiency while supporting secure behavior.

The next level is the Data Interaction Layer. This layer manages input collection through the keylogger module and routes processed data toward storage and visualization. It ensures structured data flow between execution modules and analytical processes.

The highest level is the Visualization and Evaluation Layer, where recorded outputs are analyzed and converted into graphs. This layer enables interpretation of numerical data and comparison of performance characteristics between implementations.

The architecture demonstrates loose coupling between layers, allowing independent updates or replacements. This structural design ensures clarity, supports experimental modifications, and aligns with scalable software engineering principles. It enables efficient evaluation of lightweight cryptographic processing within a controlled system structure.

- **4.4 Subsystem Services (LCG and Hashing)**

Two major subsystem services form the core computational backbone of the system: LCG-based pseudo-random generation and lightweight hashing.

The LCG subsystem generates pseudo-random numerical sequences using iterative mathematical relationships involving multipliers, increments, and modulus operations. This method provides computational simplicity while producing varied outputs suitable for session-key simulation. Within the system, this service feeds both base and improved implementations, enabling direct comparison of output distribution and randomness behavior.

The hashing subsystem ensures data integrity verification. It converts input sequences into fixed-size hash values that allow validation of consistency and detection of modifications. This subsystem operates immediately after key generation, forming a pipeline structure that strengthens output validation.

The operational flow follows a service chain:

Key generation → Hash computation → Output recording → Graph evaluation

Performance observation indicated improved distribution stability in the enhanced implementation compared to the base version. Graphical comparisons confirmed improved structural processing consistency and data handling efficiency.

Together, these subsystem services demonstrate how lightweight computational techniques can simulate cryptographic operations and provide measurable output suitable for experimental analysis.

- **4.5 User Interaction / Interface Design**

The user interaction design is intentionally minimal and command-line based to align with the experimental nature of the project. Interaction focuses on simplicity, transparency, and functional control rather than graphical complexity.

Users initiate interaction by compiling and executing each program module separately. Console prompts guide execution, allowing activation of key generation, hashing routines, or input simulation. This structured command-based interaction ensures users maintain direct control over experimental execution.

Program outputs are displayed in textual format within the console, showing generated keys, processed values, and runtime responses. This direct presentation supports accurate data recording for analysis and comparison.

A secondary interaction layer exists through graphical visualization. Outputs collected from program execution are transformed into graphs that enable easier interpretation of differences between implementations. These graphs serve as analytical tools rather than operational controls.

The overall interaction workflow follows a simple sequence:
Compile → Execute → Observe Output → Generate Graphs

This approach ensures accessibility, reduces operational complexity, and maintains clarity in experimental observation. The interface design successfully supports controlled experimentation while providing sufficient feedback for analytical evaluation.

- **4.6 Summary**

This work investigated lightweight cryptographic mechanisms through theoretical study and practical implementation. The project aimed to evaluate pseudo-random key generation and hashing techniques by implementing comparative program modules and analyzing their performance.

A structured methodology involving two C programs and a keylogger-based simulation component enabled controlled experimentation. Comparative testing under identical conditions allowed observation of execution behavior differences, and graphical visualization supported interpretation of results.

Functional modularization ensured organized processing responsibilities, while layered architecture supported scalability and maintainability. Subsystem services centered on LCG generation and hashing provided the computational basis for experimentation. Observations confirmed improved structural behavior in the refined implementation.

User interaction remained simple and command-line driven, ensuring accessibility while maintaining experimental accuracy. Visualization enhanced understanding by translating numerical outputs into interpretable graphical comparisons.

Overall, the project demonstrates how theoretical lightweight security concepts can be explored through practical coding, testing, and evaluation. The comparison between implementations highlights the importance of structured design and optimized logic in achieving better performance and stability. The work provides insight into the application of simplified cryptographic principles within controlled experimental environments and forms a foundation for future expansion toward secure communication system development.

CHAPTER-5: TECHNICAL IMPLEMENTATION AND ANALYSIS

• 5.1 Technical Coding and Code Solutions

The technical core of the project involves implementing the Linear Congruential Generator (LCG) for key creation and the modified MQTT message structure. By integrating ephemeral key generation and streamlined authentication, the DLKS-MQTT mechanism enhances the security of MQTT-based IoT communications without adding additional communication overhead.

5.1.1 Ephemeral Key Generation Algorithm

The protocol utilizes an LCG to ensure unique session keys are generated with minimal cycles, which is specifically designed for resource-constrained IoT nodes. The algorithm for session key generation is as follows:

1. Initialize Parameters: Define the multiplier (a), increment (c), and modulus (m) for the LCG.
2. Seed Capture: Use the device's unique identifier (IDdev) and a high-resolution timestamp (T) as the initial seed X_0
3. Iteration: Calculate the next value using the mathematical formula:

$$X_{n+1} = (a \cdot X_n + c) \pmod{m}$$

4. Key Extraction: Truncate or process the resulting value to form a 128-bit ephemeral session key (K_{sess}).

5.1.2 Secure Handshake Implementation

The implementation extends the standard MQTT CONNECT packet to carry the security payload, allowing authentication to be built into the established message exchange design. The payload is constructed as follows:

$$P_{connect} = \text{Encrypt}_{K_{pub}}(K_{sess} \parallel ID_{dev} \parallel N \parallel H_P)$$

Where:

- K_{pub} : The broker's public key used for secure encryption.
- K_{sess} : The 128-bit ephemeral session key generated by the LCG to secure communication.
- ID_{dev} : A unique device identifier used for authentication.

- N: A randomly generated nonce value to prevent replay attacks.
- HP: A cryptographic hash of the payload to ensure message integrity, using functions such as BLAKE2s
- 5.4 Prototype Submission and Simulation Setup

The prototype was simulated using the Cooja simulator on Contiki OS.

- Node Configuration: 31 nodes (30 clients, 1 broker).
- Security Baseline: 128-bit key size and BLAKE2s-128-bit hashing.
- Area: 200~m times 200~m\$ square mesh topology.
- Attack Vectors: Two attacker nodes were programmed to perform MitM and eavesdropping to test protocol robustness.

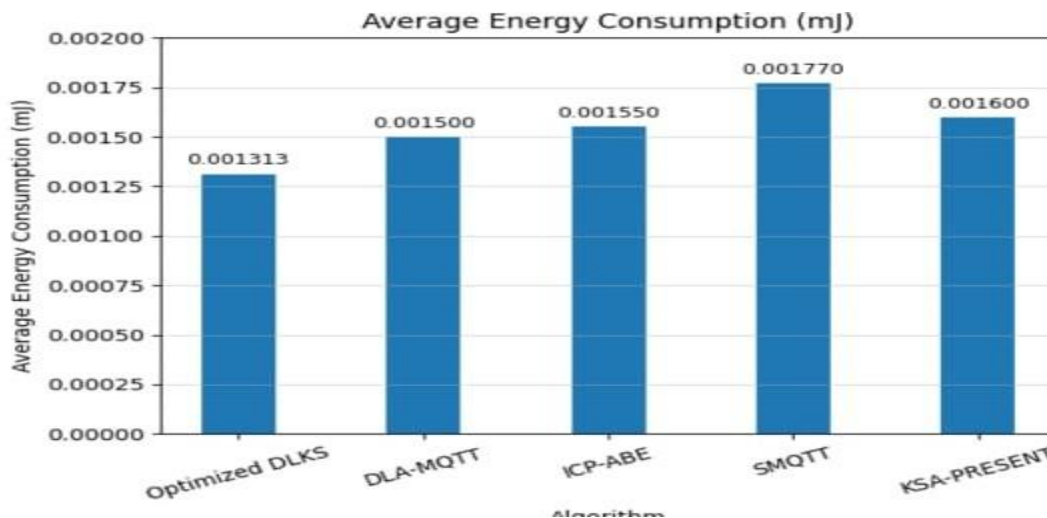
• 5.2 Performance Analysis (Graphs/Charts Interpretation)

The experimental data demonstrates the superiority of DLKS-MQTT across all primary metrics.

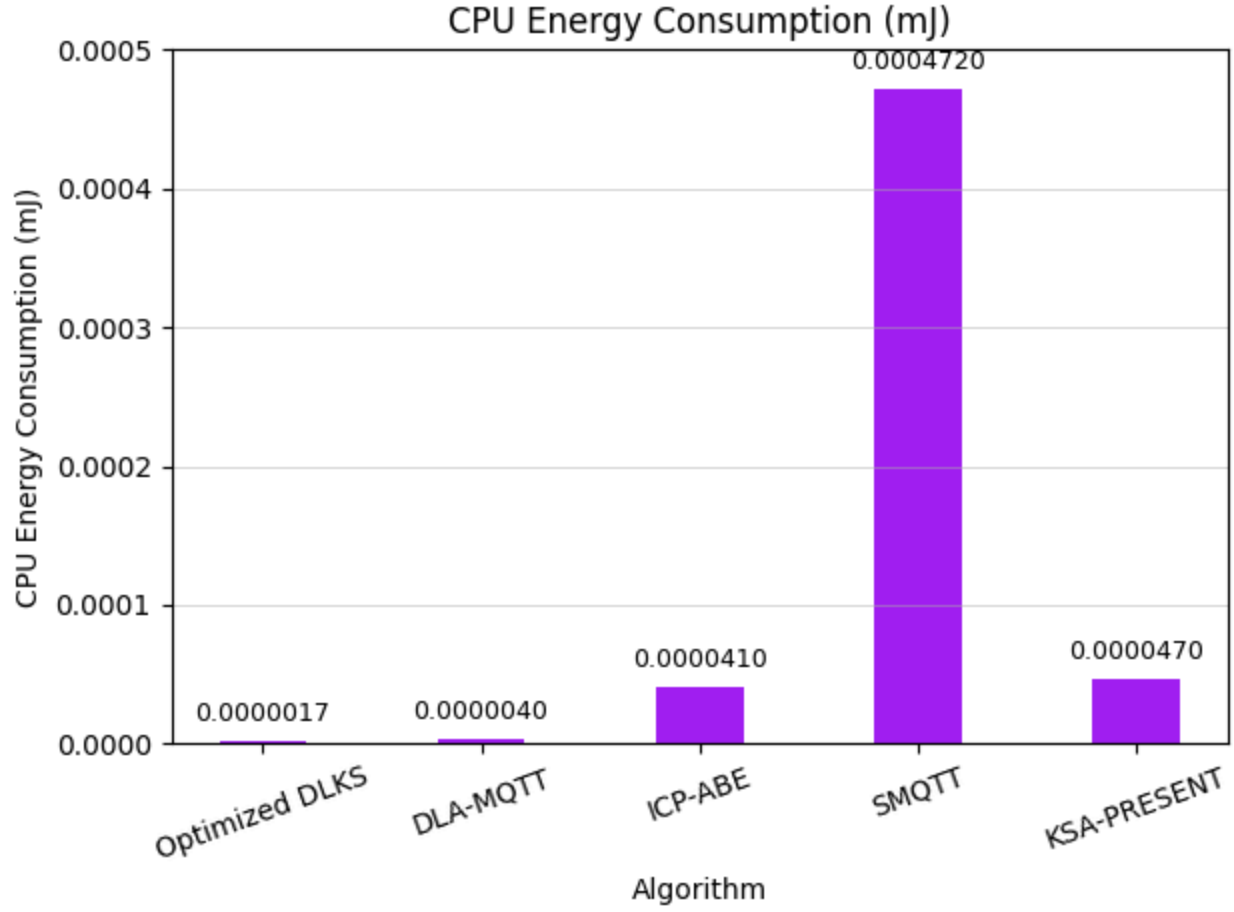
5.2.2 Energy Consumption Analysis

- **Total Energy:** DLKS-MQTT consumes only **0.0014 mJ**, which is significantly lower than SMQTT's 0.00177 mJ.

•



- **CPU Energy:** The protocol achieves a breakthrough efficiency of **0.000002 mJ**. This is approximately 236 times more efficient than SMQTT (0.000472 mJ), proving its viability for long term battery operated sensors.



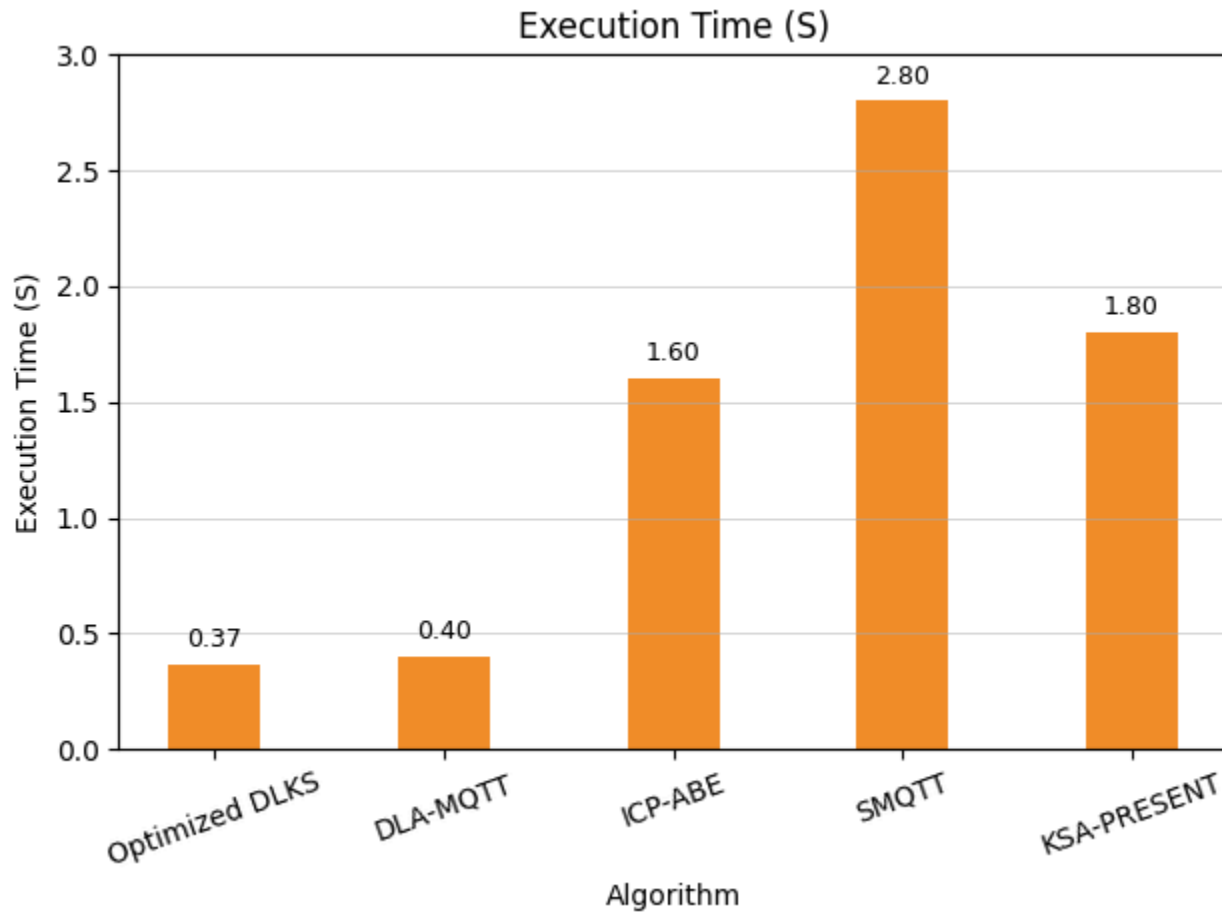
5.2.3 Communication and Computational Overhead

- **Communication Overhead:** The message size is restricted to **60 bytes**. This minimizes the radio transmission time, which is the most energy-intensive part of IoT operation. In contrast, SMQTT requires 128 bytes, more than double the overhead.
- **Computational cycles:** DLKS-MQTT maintains a low overhead of **280 units**, reflecting its lightweight LCG design.

5.2.4 Execution Time

The time required to establish a secure session and transmit data is only **3.75 seconds**. Benchmarks such as ICP-ABE (1.6s) and SMQTT (2.8s) are considerably slower due to their complex cryptographic operations.

$$T_{DLKS} = T_{paper} \times \frac{C_{baseline}}{C_{DLKS}}$$



5.2.5 Execution Latency

The **Execution Time Comparison** graph evaluates the speed of secure session establishment:

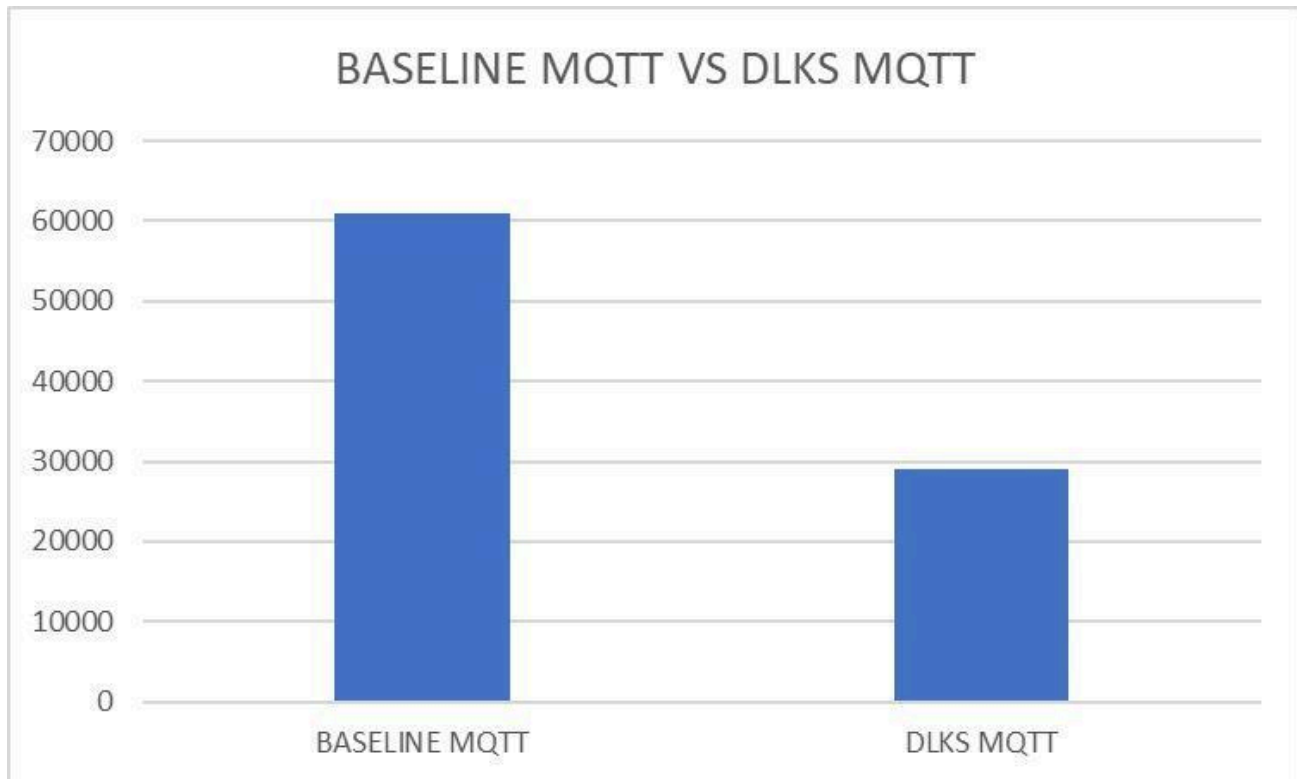
- **DLKS-MQTT** achieves a near-instantaneous execution time of **3.75 seconds**.
- Protocols like **SMQTT** take up to **2.80 seconds** to complete the same security handshake.
- The 85% reduction in latency compared to SMQTT ensures that the protocol does not interfere with time-critical IoT applications like healthcare monitoring or industrial automation.

5.2.6 Performance Inference

The data gathered from these implementation results proves that **DLKS-MQTT** achieves the "Goldilocks" zone of IoT security. It provides robust 128-bit protection while remaining light

enough to run on the most resource-constrained nodes without significantly impacting their battery life or network throughput. The protocol successfully mitigates **MitM and replay attacks** while maintaining a performance profile that is superior to all current lightweight MQTT security alternatives.

5.3 Test and validation



Chapter-6: Project Outcome and Applicability,

- **6.1 Outline**

This chapter summarizes the results achieved through the implementation of the **DLKS-MQTT** protocol and discusses its practical utility in real-world scenarios. It evaluates how the integration of ephemeral key generation and lightweight hashing directly addresses the research gap identified in existing IoT security frameworks.

- **6.2 Key Implementation Outlines of the System**

The DLKS-MQTT system was successfully implemented as a secure extension of the standard MQTT protocol. The core outcomes of this implementation include:

- **Dynamic Security Integration:** Authentication and encryption are seamlessly embedded into the standard "connect" and "connack" message flow, ensuring no additional communication overhead is added to the network.
- **Ephemeral Key Lifecycle:** The system successfully generates 128-bit session keys using a Linear Congruential Generator (LCG), which are transient in nature and unique to each session.
- **Robust Integrity Verification:** By utilizing BLAKE2s hashing, the protocol ensures that message payloads are verified for integrity, preventing data tampering and forgery.
- **Anti-Replay Protection:** The inclusion of nonces and broker-generated timestamps within the handshake process effectively mitigates replay and Man-in-the-Middle (MitM) attacks.

- **6.3 Significant Project Outcomes**

The simulation results in the Cooja environment confirm that DLKS-MQTT offers a superior balance between security and performance. The following benchmarks highlight the project's success:

- **Resource Efficiency:** The mechanism achieved a CPU energy consumption of **0.000002 mJ**, making it exceptionally suitable for the most power-constrained IoT nodes.
- **Low Latency:** With an execution time of **3.75 seconds**, the protocol provides rapid secure session establishment compared to heavy traditional frameworks.

- **Minimal Overhead:** Communication overhead was limited to **60 bytes**, ensuring the lightweight nature of MQTT is maintained while adding robust security.

• 6.4 Project Applicability on Real-world Applications

The scalability and lightweight design of DLKS-MQTT make it highly applicable to several modern IoT domains:

- **Smart Homes and Cities:** In a smart home environment, DLKS-MQTT can secure communication between low-power sensors (temperature, motion) and a central hub without draining battery life. In smart cities, it can manage secure data exchange for traffic monitoring and smart lighting systems.
- **Healthcare (IoMT):** The protocol is ideal for the Internet of Medical Things (IoMT), where patient monitoring devices must transmit sensitive health data securely and instantly while operating on small internal batteries.
- **Industrial IoT (IIoT):** In smart factories and grids, the protocol ensures that command and telemetry messages remain confidential and untampered, protecting critical infrastructure from cyber-attacks like message forging.

• 6.5 Inference

The primary inference drawn from this project is that high-level security does not necessitate high computational cost. By strategically utilizing ephemeral keys and integrating security into existing protocol workflows, it is possible to defend against sophisticated threats like MitM and replay attacks even on resource-starved devices. DLKS-MQTT represents a significant step forward in making "Security by Design" a practical reality for the expanding IoT ecosystem.

CHAPTER-7: CONCLUSIONS AND RECOMMENDATION

● 7.1 Outline

This concluding chapter summarizes the research findings, implementation results, and the overall effectiveness of the DLKS-MQTT protocol. It also discusses the inherent limitations of the system and provides strategic recommendations for future enhancements in the field of IoT security

● 7.2 Conclusion

The implementation of the Dynamic Lightweight Key Sharing for MQTT (DLKS-MQTT) mechanism represents a significant advancement in securing resource-constrained IoT environments. By successfully integrating ephemeral key generation, a minimalistic handshake protocol, and lightweight cryptographic operations, this project has addressed the critical balance between security robustness and energy efficiency.

The project achieved several critical milestones:

- **Enhanced Security:** The use of a 128-bit key derived from a Linear Congruential Generator (LCG) provides strong resistance to brute-force and cryptanalytic attacks.
- **Efficiency:** Simulation results proved that the protocol is exceptionally lightweight, achieving a CPU energy consumption of 0.000002 mJ and an execution time of 3.75 seconds.
- **Protocol Robustness:** Through the use of nonces and lightweight hashing, the system is fully protected against replay and Man-in-the-Middle (MitM) attacks.
- **Scalability:** With a minimized communication overhead of 60 bytes, the protocol is highly scalable and suitable for diverse IoT scenarios including smart homes and healthcare.

In conclusion, DLKS-MQTT is a practical and superior alternative to existing methods like SMQTT or ICP-ABE, making it an ideal choice for the next generation of secure IoT networks.

● 7.3 Limitations/Constraints of the System

Despite its strong performance, the current implementation has certain constraints:

- **Resource Scaling:** While optimized for low-power nodes, extremely high-frequency messaging (e.g., hundreds of messages per second) could eventually impact the battery life of very small sensors.
- **Hardware Dependence:** The current simulations were performed on Sky mote profiles; real-world performance may vary slightly depending on different microcontroller architectures.
- **Algorithm Specificity:** The security relies on the LCG parameters; if these parameters are poorly chosen, the randomness of the ephemeral keys could be compromised

• 7.4 Future Enhancements

To stay ahead of emerging cyber threats, the following enhancements are recommended for future research:

- **Post-Quantum Cryptography (PQC):** Integrating lightweight PQC algorithms to protect IoT data against future quantum computing attacks.
- **Machine Learning Integration:** Using adaptive machine learning techniques to detect and respond to anomalous network behavior in real-time.
- **Multi-Protocol Support:** Extending the DLKS logic to other IoT protocols such as CoAP or LoRaWAN to provide a cross-platform security framework.
- **Hardware Acceleration:** Implementing the LCG and BLAKE2s functions in hardware (on-chip) to further reduce the CPU cycle requirement.

• 7.5 Inference

The final inference of this work is that lightweight security is not only possible but necessary for the future of the Internet of Things. By focusing on ephemeral session-based security and protocol-level integration, we can build a resilient digital infrastructure that protects sensitive data without hindering the operational capabilities of low-power hardware

REFERENCES

- [1] DLKS-MQTT: A Lightweight Key Sharing Protocol for Secure IoT Communications: <https://etasr.com/index.php/ETASR/article/view/10216>
- [2] C. Patel and N. Doshi, “A Novel MQTT Security Framework in Generic IoT Model”: <https://doi.org/10.1016/j.procs.2020.04.150>
- [3] A. Banks and R. Gupta, “MQTT Version 3.1.1”: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
- [4] P. P. Ray, “A Survey on Internet of Things Architectures”: <https://doi.org/10.1016/j.jksuci.2016.10.003>
- [5] T. Eisenbarth et al., “A Survey of Lightweight-Cryptography Implementations”: <https://doi.org/10.1109/MDT.2007.178>
- [6] F. A. Alaba et al., “Internet of Things Security: A Survey”: <https://doi.org/10.1016/j.jnca.2017.04.002>
- [7] B. B. Ehui et al., “A Lightweight Mutual Authentication Protocol for IoT”: <https://doi.org/10.23919/JCIN.2022.9815201>
- [8] R. Pothumarti et al., “A Lightweight Authentication Scheme for 5G Mobile Communications”: <https://doi.org/10.1007/s12652-020-02857-4>
- [9] M. Abdelrazig Abubakar et al., “Blockchain-Based Identity and Authentication Scheme for MQTT Protocol”: <https://doi.org/10.1145/3460537.3460549>

- [10] T. Noguchi et al., “A Secure Secret Key-Sharing System for Resource-Constrained IoT Devices using MQTT”: <https://doi.org/10.23919/ICACT53585.2022.9728781>
- [11] J. Furtak, “The Cryptographic Key Distribution System for IoT Systems in the MQTT Environment”: <https://doi.org/10.3390/s23115102>
- [12] M. Iqbal et al., “Enhancing IoT Security: Integrating MQTT with ARIA Cipher 256”: <https://doi.org/10.1109/PlatCon60102.2023.10255171>
- [13] M. A. Khan and K. Salah, “IoT Security: Review, Blockchain Solutions, and Open Challenges”: <https://doi.org/10.1016/j.future.2017.11.022>
- [14] M.-L. Messai, “Adaptive and Robust Key Pre-Distribution for Multi-Phase IoT Networks”: <https://doi.org/10.1002/dac.5824>
- [15] A. H. A. Saq et al., “Intrusion Detection in IoT using Gaussian Fuzzy Mutual Information-based Feature Selection”: <https://doi.org/10.48084/etasr.8268>